BHARATIYA VIDYA BHAVAN'S
SARDAR PATEL INSTITUTE OF TECHNOLOGY
Bhavan's Campus, Munshi Nagar, Andheri (West), Mumbai – 400058-India

DEPARTMENT OF COMPUTER ENGINEERING

Course name: LLM Course Code: CE453 Sem: VII A.Y. 2026-27

DETAILS	
Name	Anurag Akhilesh Singh
UID no.	2023300231
Batch	2

EXPERIMENT NO. 7	
AIM :	To prepare the Medical Support RAG application from Experiment 5 for API-based use by adding input validation, error handling, basic logging, response-time measurement, and endpoint testing.
PROBLEM STATEMENT / OBJECTIVES :	The Medical Support RAG application developed in Experiment 5 provides question-answering functionality by retrieving relevant information from a predefined healthcare knowledge base and generating answers using a Large Language Model (LLM). However, to make the application accessible to external clients and suitable for integration with other applications, it must expose its functionality through a structured API and handle requests reliably. A production-oriented API requires input validation, controlled error handling, response-time measurement, source attribution, and systematic testing. It must also handle invalid inputs and questions that cannot be answered from the available documents without generating unsupported information. Therefore, this experiment extends the existing RAG application by developing a FastAPI-based interface, introducing request validation and error handling, measuring response latency, and evaluating the system through automated and live HTTP tests. **Objectives :** 1. To convert the RAG application developed in Experiment 5 into an API-accessible application using FastAPI. 2. To expose endpoints for checking application health and submitting questions to the RAG pipeline. 3. To validate incoming requests and reject empty, whitespace-only, oversized, missing, null, or incorrectly typed questions. 4. To retrieve relevant context from the healthcare knowledge base before generating an answer. 5. To return structured responses containing the generated answer, relevant source metadata, and response-time information.

	6. To implement controlled error handling for invalid requests and generation-backend failures. 7. To handle out-of-domain questions using an insufficient-information fallback when the available context does not support an answer. 8. To record request latency and basic application logs for monitoring and debugging. 9. To test the API using valid questions, invalid inputs, out-of-domain queries, and response-structure checks. 10. To evaluate the application using observable test results and identify areas requiring further improvement.
THEORY :	**1. Retrieval-Augmented Generation (RAG) :** Retrieval-Augmented Generation is an approach that combines information retrieval with Large Language Model generation. Instead of relying exclusively on the model's pretrained knowledge, the application retrieves relevant passages from a designated knowledge base and supplies them as context for answering a user's question. The major stages of a RAG pipeline are: ● Document loading: Information is extracted from source documents such as text files, PDFs, and Word documents. ● Text splitting: Large documents are divided into smaller chunks suitable for embedding and retrieval. ● Embedding generation: Each chunk is converted into a numerical vector representing its semantic meaning. ● Vector storage: The vectors and associated document metadata are stored in a vector database. ● Query embedding and retrieval: The user's question is converted into a vector, and the vector database retrieves semantically similar passages. ● Context-grounded generation: The retrieved passages are provided to the LLM to generate an answer based on the available evidence. ● Source attribution and fallback: Source metadata is returned where available, and the application can decline to answer when sufficient supporting information is not found. In this experiment, the healthcare RAG pipeline uses Chroma as the vector store and the sentence-transformers/all-MiniLM-L6-v2 embedding model, as configured in the project. **2. FastAPI** FastAPI is a Python framework used to build HTTP APIs. It supports request

validation, structured responses, automatic OpenAPI schema generation, and interactive API documentation.

The application exposes the following principal endpoints:

- GET /health — checks the availability of the API.
- POST /ask — accepts a question, executes the RAG pipeline, and returns the response.

Interactive documentation is available through the /docs route when the server is running.

3. Input Validation

Input validation ensures that requests conform to the expected schema before they reach the retrieval and generation pipeline. In this application, the question field must contain a valid, non-blank string within the configured length limit. Invalid requests, including missing fields, empty questions, whitespace-only questions, oversized inputs, and incorrect data types, are expected to be rejected with an appropriate client-error response. The tested validation responses use HTTP 422.

4. Error Handling

Errors can occur because of invalid input, unavailable generation services, network problems, timeouts, or configuration issues. Controlled error handling prevents avoidable application crashes and helps return meaningful error responses without exposing API keys, credentials, or internal stack traces. Backend-failure handling should be verified separately by simulating a genuine failure. A successful health check alone does not establish that the generation backend is available.

5. Response-Time Measurement and Logging

Response-time measurement records how long an API request takes to process. The application records latency in milliseconds to support performance observation and troubleshooting. Client-side latency measures the total time observed by the test client, whereas server-side latency measures the time recorded by the application. These values may differ because of network overhead, request processing, and response transmission.
Basic logging can be used to record relevant application events and errors without logging secrets or unnecessary sensitive information.

6. Out-of-Domain Handling

A question is out of domain when it cannot be answered using the available

knowledge base. For a healthcare RAG application, unrelated questions about topics such as geography, sports, or quantum computing should not be answered using unrelated retrieved passages. The application uses an insufficient-information fallback when the available context does not support a response. This behavior reduces the risk of unsupported answers, although it does not independently guarantee that every generated answer is factually correct.

7. Testing Methodology

The application is evaluated using live HTTP integration tests. These tests send requests to the running API and compare the observed results with predefined expectations.

The tests cover:

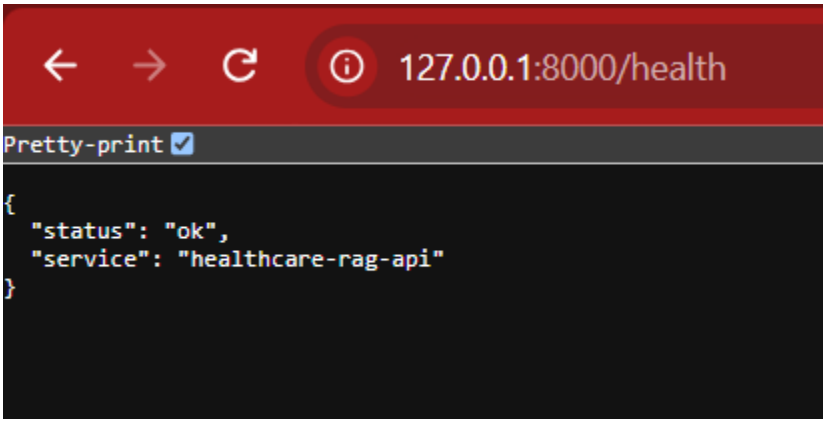
- API availability and documentation.
- Valid healthcare questions.
- Invalid and malformed requests.
- Out-of-domain questions and fallback behavior.
- Script-like and SQL-like text inputs.
- Response structure and source metadata.
- Response latency.

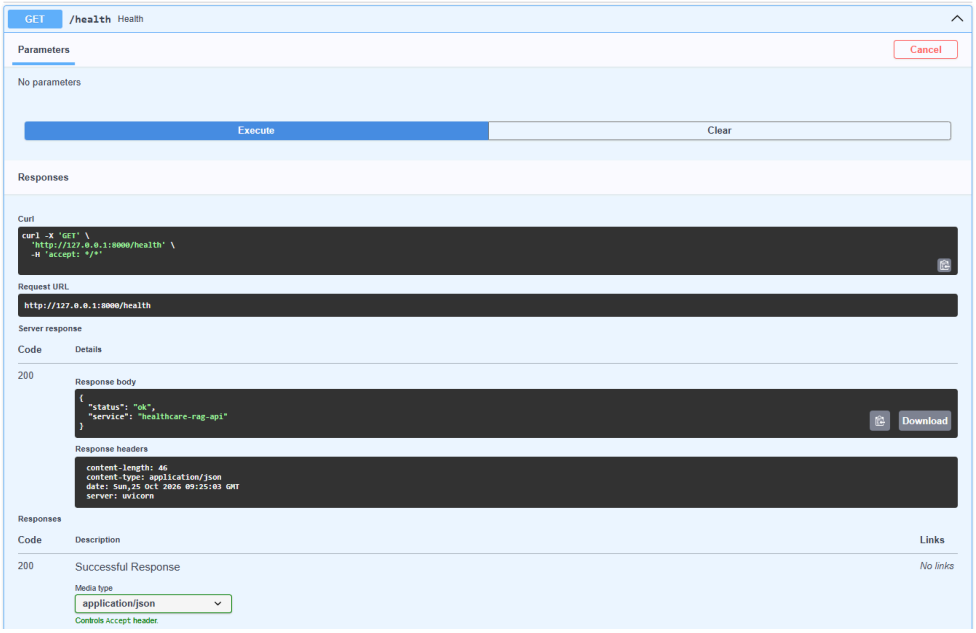
A test is marked PASS when the observed behavior satisfies its defined test criteria. A test is marked FAIL when the observed behavior does not satisfy those criteria. An HTTP 200 response alone does not prove semantic correctness; answers must also be checked against their source documents.

CODE :

SCREENSHOTS :

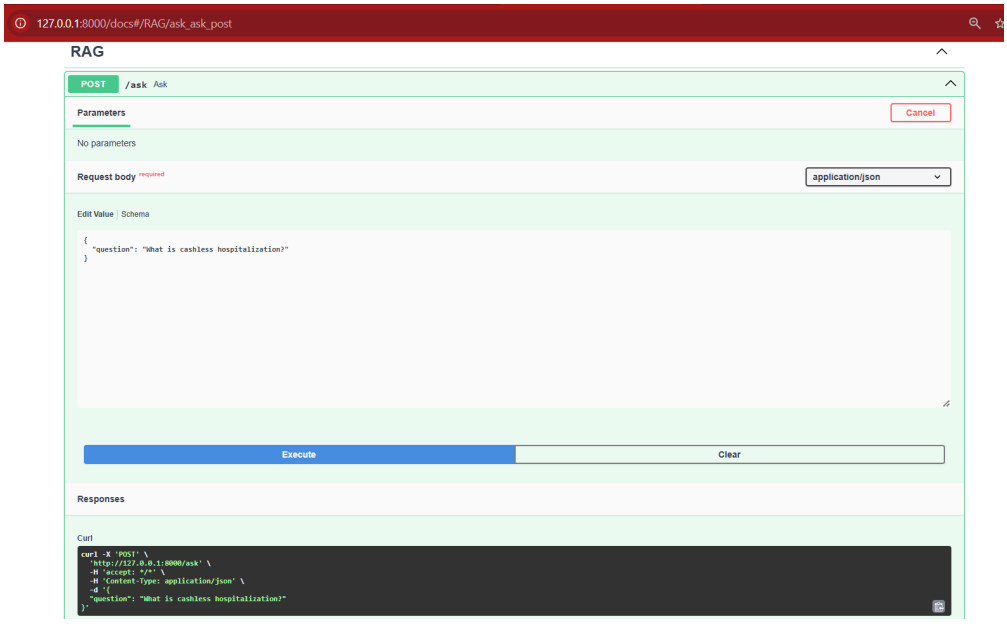
1. GET /health :

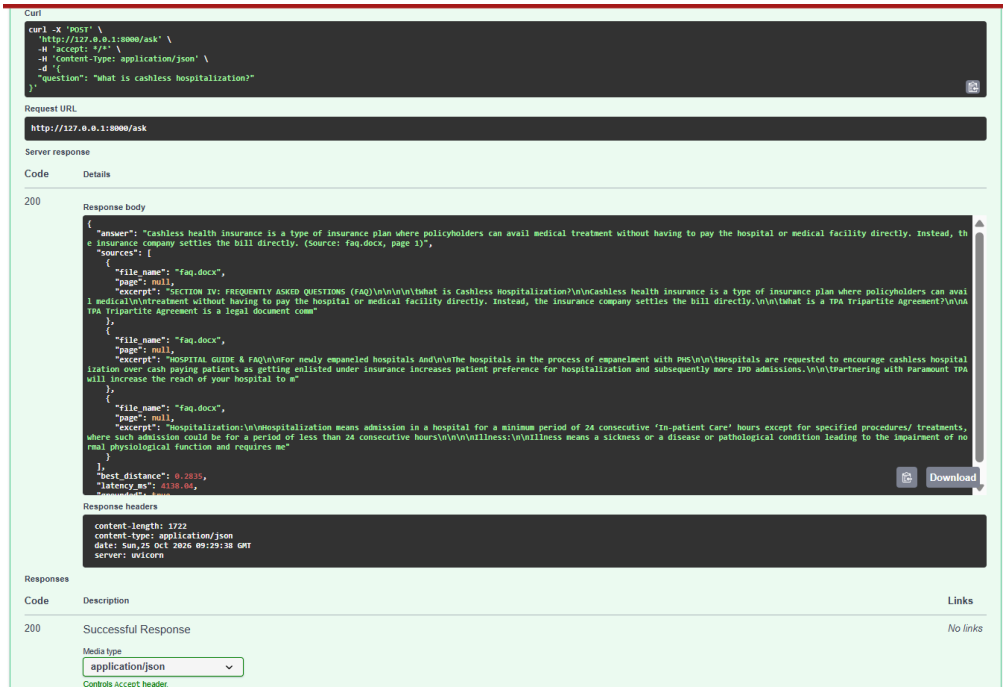




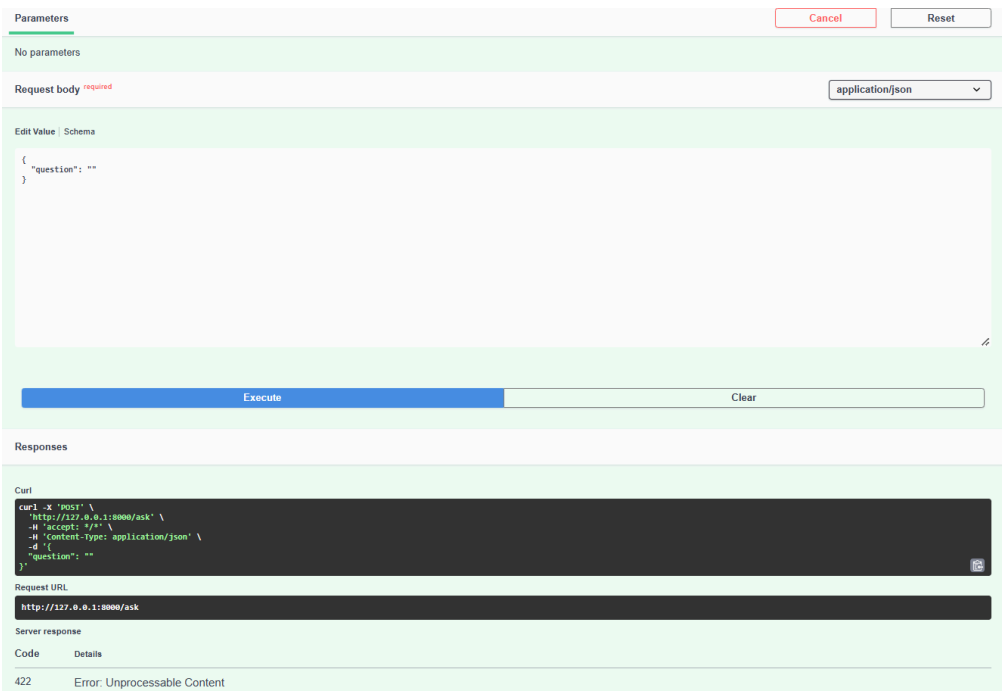
2. API Test :

a. Supported Question :

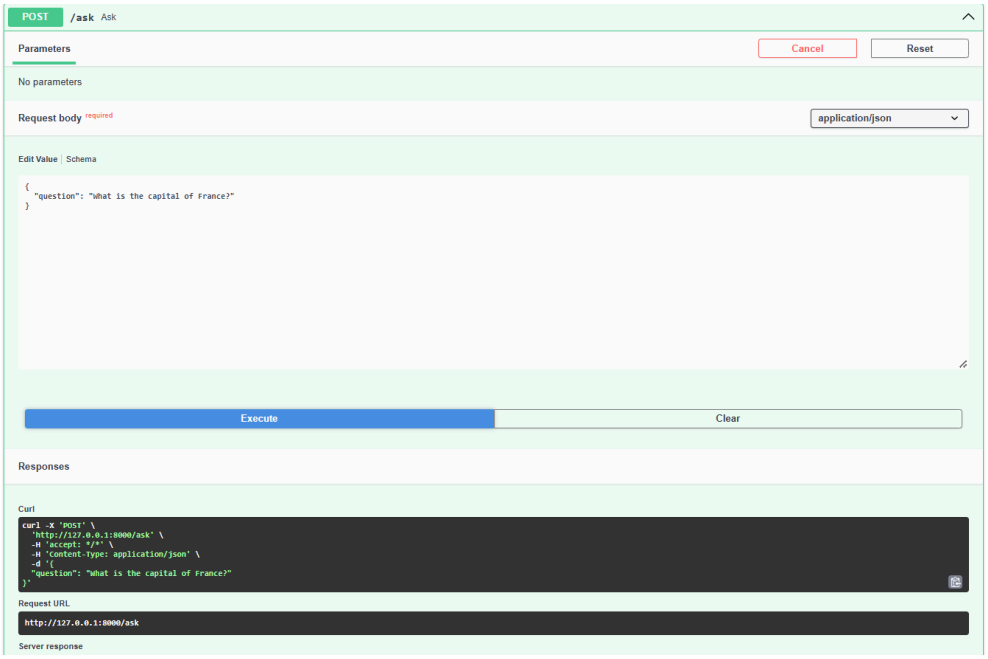


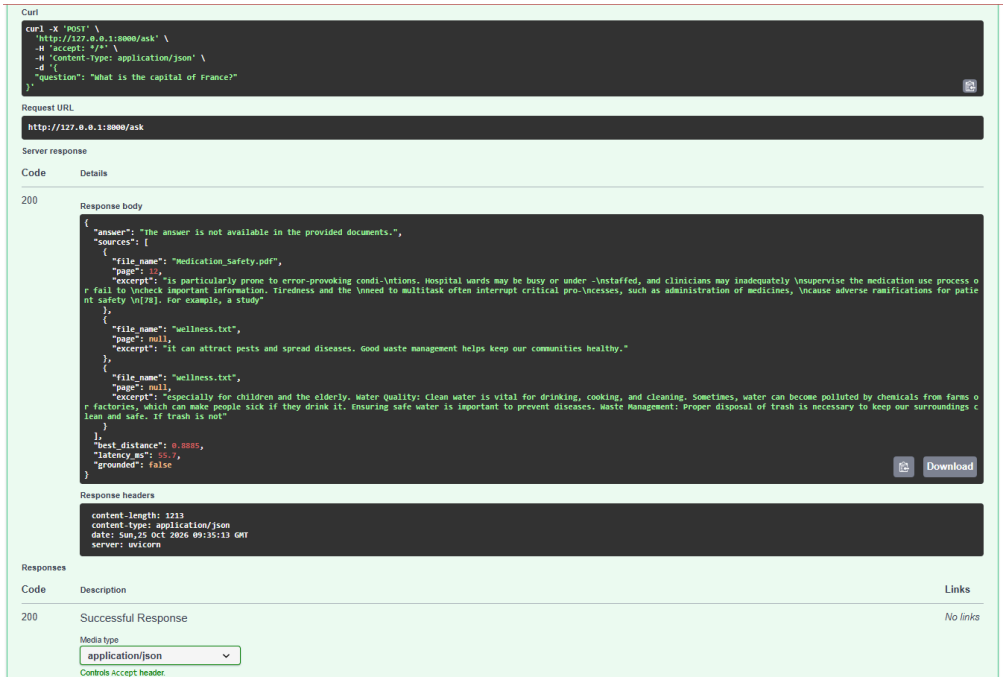


b. Empty Question :



c. Unsupported Question :





OBSERVATIONS:	The application was evaluated using a live HTTP integration test suite. The recorded report contains 25 tests covering API availability, request validation, relevant knowledge-base questions, out-of-domain questions, security-like input handling, and response-structure checks.

A. Summary of the recorded test run :

Test Category	Total Tests	Passed	Failed
Health and API documentation	2	2	0
Input validation	7	7	0
Relevant knowledge-base questions	9	9	0
Out-of-domain questions	5	5	0
Script/SQL-like input handling	1	1	0
Response structure and source metadata	1	1	0
Total	25	25	0

Overall recorded result: 25/25 tests passed. The complete suite successfully executed, with all relevant questions returning grounded answers and valid source metadata.

B. Observation table:

Test Case / Condition	Expected Result	Actual Result Obtained (From JSON)	Client Latency (ms)	Pass / Fail
GET /health returns HTTP 200 and status=ok	API reports that the service is running	HTTP 200; response={"status": "ok", "service": "healthcare-rag-api" }	22.54	PASS

	OpenAPI schema is available	API schema is available	HTTP 200; API docs should be at http://127.0.0.1:8000/docs	23.71	PASS
	Empty question	HTTP 422 validation error	HTTP 422; expected HTTP 422; body={"detail":[{"type":"string_too_short"...	23.32	PASS
	Whitespace-only question	HTTP 422 validation error	HTTP 422; expected HTTP 422; body={"detail":[{"type":"value_error"...	14.31	PASS
	Question exceeds 1000 characters	HTTP 422 validation error	HTTP 422; expected HTTP 422; body={"detail":[{"type":"string_too_long"...	15.37	PASS
	Missing required question field	HTTP 422 validation error	HTTP 422; expected HTTP 422; body={"detail":[{"type":"missing"...	16.05	PASS

	Question supplied as number	HTTP 422 validation error	HTTP 422; expected HTTP 422; body={"detail":[{"type":"string_type"...	28.87	PASS
	Null question	HTTP 422 validation error	HTTP 422; expected HTTP 422; body={"detail":[{"type":"string_type"...	15.74	PASS
	Malformed JSON body is rejected	Client error response	HTTP 422; expected HTTP 422; body={"detail":[{"type":"json_invalid"...	5.03	PASS
	Relevant KB question 01: What does the wellness note say about hydration?	Grounded answer with relevant source metadata	HTTP 200; grounded=True; source_files=['wellness.txt']; answer=According to...	2290.64	PASS
	Relevant KB question 02: What	Grounded answer with relevant	HTTP 200; grounded=True; source_files=['wellness.txt'];	4593.84	PASS

	are the signs of under-nutrition?	source metadata	answer=The signs of...		
	Relevant KB question 03: What are some healthy-living tips regarding proteins?	Grounded answer with relevant source metadata	HTTP 200; grounded=True; source_files=['wellness.txt']; answer=According to...	12075.91	PASS
	Relevant KB question 04: What is the turnaround time for cashless file submission to the TPA?	Grounded answer with relevant source metadata	HTTP 200; grounded=True; source_files=['faq.docx']; answer=Hospitals should...	1533.38	PASS
	Relevant KB question 05: What is the Provider	Grounded answer with relevant source metadata	HTTP 200; grounded=True; source_files=['faq.docx']; answer=The Provider Module...	1738.66	PASS

	Module?				
	Relevant KB question 06: Where should the hospital submit claim documents physically?	Grounded answer with relevant source metadata	HTTP 200; grounded=True; source_files=['faq.docx']; answer=The hospital can...	1535.77	PASS
	Relevant KB question 07: What is the National Health Claim Exchange (NHCX)?	Grounded answer with relevant source metadata	HTTP 200; grounded=True; source_files=['faq.docx']; answer=The National Health...	2250.99	PASS
	Relevant KB question 08: How is a medication error defined in the medication-safety	Grounded answer with relevant source metadata	HTTP 200; grounded=True; source_files=['Medication_Safety.pdf']; answer=A med...	11982.17	PASS

	document?				
	Relevant KB question 09: What is polypharmacy?	Grounded answer with relevant source metadata	HTTP 200; grounded=True; source_files=['Medication_Safety.pdf']; answer=In its...	1973.05	PASS
	Out-of-domain question 01: What is the capital of France?	Insufficient-information fallback	HTTP 200; grounded=False; fallback_detected=True	42.40	PASS
	Out-of-domain question 02: Who won the FIFA World Cup in 2022?	Insufficient-information fallback	HTTP 200; grounded=False; fallback_detected=True	33.31	PASS
	Out-of-domain question 03: How do you build a quantum computer?	Insufficient-information fallback	HTTP 200; grounded=False; fallback_detected=True	15.40	PASS

	Out-of-domain question 04: What is the recipe for chocolate chip cookies?	Insufficient-information fallback	HTTP 200; grounded=False; fallback_detected=True	1616.32	PASS
	Out-of-domain question 05: Explain the plot of a popular science-fiction movie.	Insufficient-information fallback	HTTP 200; grounded=False; fallback_detected=True	53.56	PASS
	Script/SQL-like text does not crash the API	API remains stable and treats input as text	HTTP 200; API returned structured response; grounded=False	34.22	PASS
	Answer response contains required fields and valid source metadata	Required response fields and valid source metadata	HTTP 200; missing_fields=[]; source_shape_valid=True	1958.49	PASS

C. Relevant knowledge-base question results :

No.	Question	Actual Result	Backend Latency (ms)	Pass / Fail
1	What does the wellness note say about hydration?	Grounded answer returned from wellness.txt	2,265.26	PASS
2	What are the signs of under-nutrition?	Grounded answer returned from wellness.txt	4,589.86	PASS
3	What are some healthy-living tips regarding proteins?	Grounded answer returned from wellness.txt	12,044.34	PASS
4	What is the turnaround time for cashless file submission to the TPA?	Grounded answer returned from faq.docx	1,523.31	PASS
5	What is the Provider Module?	Grounded answer returned from faq.docx	1,729.21	PASS

6	Where should the hospital submit claim documents physically?	Grounded answer returned from faq.docx	1,527.72	PASS
7	What is the National Health Claim Exchange (NHCX)?	Grounded answer returned from faq.docx	2,221.80	PASS
8	How is a medication error defined in the medication-safety document?	Grounded answer returned from Medication_Safety.pdf	11,954.77	PASS
9	What is polypharmacy?	Grounded answer returned from Medication_Safety.pdf	1,952.50	PAS

D. Out-of-domain question results

Five unrelated questions were tested: the capital of France, the winner of the 2022 FIFA World Cup, building a quantum computer, a chocolate-chip-cookie recipe, and a science-fiction movie plot. All five tests detected the configured fallback and reported grounded=False. Each test was therefore marked PASS according to the fallback criterion used in the live test report.

	E. Interpretation of observations The health endpoint, OpenAPI schema, input-validation checks, out-of-domain fallback checks, script/SQL-like input handling, and response-contract test passed according to the recorded criteria. Two relevant-question tests failed because the application returned the insufficient-information fallback instead of the expected answer. The source filenames were listed in the report, but this alone does not prove that the correct supporting passages were retrieved or accepted by the grounding logic. The retrieved chunks, similarity distances, document contents, and fallback threshold must be examined to determine the cause. The measured response times varied across requests. These values represent one recorded run and may vary with model initialization, embedding-model warm-up, network conditions, and the selected generation backend.
CONCLUSION :	I successfully extended the Medical Support RAG application developed in Experiment 5 by integrating a FastAPI interface for API-based question answering. I implemented input validation, structured responses, source metadata, response-time measurement, out-of-domain fallback handling, and live HTTP integration testing. During the final recorded test run, I executed 25 tests, all of which passed. I verified the API health endpoint, OpenAPI schema availability, input validation for invalid requests, retrieval and generation of answers for relevant healthcare questions, fallback handling for out-of-domain queries, script/SQL-like input handling, and response structure with valid source metadata. Through this experiment, I gained practical experience in preparing a RAG application for API-based use and evaluating its functionality through reproducible integration tests. The results demonstrated that the application satisfied the defined test criteria during the recorded run, while manual verification of generated answers against the original knowledge-base documents remained important for confirming semantic correctness.